Competitor Price Scraper and Dynamic Repricer

Market Research and Product Requirements

A pricing intelligence system for monitoring competitor prices, matching equivalent products, protecting margins and supporting controlled repricing.

MONITOR	MATCH	CALCULATE	PROTECT
Competitor prices	Products and variants	Pricing recommendations	Margin safeguards

Research coverage

Market research	Competitor products, customer problems and published pricing models
Data collection	Scraping methods, candidate platforms and source-access restrictions
Product requirements	Workflows, features, pricing rules and acceptance criteria
Technical design	Technology stack, architecture, database, APIs, deployment and security

Target customers	Mid-market direct-to-consumer merchants
Primary market	India, with international platforms considered for future expansion

Contents

I Market Research **1**

1 Executive Summary **1**
1.1 Research limitations . . . 1

2 Project Brief and Scope **1**
2.1 Original problem statement . . . 1
2.2 Research questions . . . 1
2.3 Evidence classification . . . 2

3 Competitive Landscape **2**
3.1 Existing products . . . 2
3.2 Common market capabilities . . . 2
3.3 Customer problems reported in reviews . . . 3

4 Indian Market and Potential Customers **3**
4.1 Potential customer segments . . . 3
4.2 Candidate source categories . . . 3

5 Pricing Strategies and Margin Protection **4**
5.1 Common pricing strategies . . . 4
5.2 Gross margin versus markup . . . 4
5.3 Competitor-pricing example . . . 4
5.4 Additional pricing considerations . . . 4

6 Web-Scraping and Data Feasibility **5**
6.1 Potential extraction methods . . . 5
6.2 Potential observation fields . . . 5
6.3 Product matching . . . 5
6.4 Failure handling and freshness . . . 5
6.5 Source access restrictions . . . 6

7 Competitor Pricing and Business Models **6**

8 Research Findings and Opportunities **6**
8.1 Product matching . . . 6
8.2 Data freshness . . . 6
8.3 Explainable pricing . . . 7
8.4 Safe repricing . . . 7
8.5 Research conclusion . . . 7

9 Source Strategy and Feasibility **7**

9.1 Merchant data and competitor data . . . 7
9.2 Candidate source inventory . . . 7

II Product Requirements Document 8

10 Product Overview 8
10.1 Problem statement . . . 8
10.2 Product goals . . . 8
10.3 Non-goals . . . 8

11 Users and Permissions 8

12 Primary User Workflows 9
12.1 Workflow A: Configure monitoring . . . 9
12.2 Workflow B: Review a price change . . . 9
12.3 Workflow C: Publish a price . . . 9

13 Feature Scope and Priorities 9

14 Platform and Source Strategy 10
14.1 Merchant-store integrations . . . 10
14.2 Competitor source inventory . . . 11

15 Functional Requirements 11

16 Recommended Technology Stack 13

17 System Architecture 13
17.1 Architecture overview . . . 13
17.2 System architecture diagram . . . 13
17.3 Architecture principles . . . 14
17.4 Component responsibilities . . . 14
17.5 Price collection and repricing flow . . . 15

18 Recommended Project Structure 15

19 Deployment Architecture 16
19.1 Environment variables . . . 17

20 Proposed Data Model 17

21 API Requirements 18

22 Pricing Engine Requirements 18
22.1 Calculation inputs . . . 18
22.2 Minimum safe selling price . . . 19

22.3 Recommendation algorithm 19
22.4 Publishing safeguards 19

23 Dashboard and Interface Requirements **20**

24 Non-Functional Requirements **20**

25 Security and Operational Controls **21**
25.1 Authentication and authorisation 21
25.2 URL and network security 21
25.3 Credential handling 21
25.4 Operational monitoring 21

26 Acceptance Criteria **21**

27 Product Metrics **22**

28 Risks and Mitigations **23**

29 Implementation Sequence **23**

30 Decisions Required Before Implementation **24**

A Source Register **24**

Part I
Market Research

1 Executive Summary

The proposed product is a Competitor Price Scraper and Dynamic Repricer for mid-market direct-to-consumer (D2C) merchants.

It will collect competitor price observations, match comparable products, maintain price history, calculate selling-price recommendations and protect the merchant's configured minimum margin.

Research identified existing products including PriceSync by Jalpi, PriceZen, Databitcent, Prisync, Price2Spy and PriceIntel AI. They offer different combinations of competitor monitoring, matching, alerts, analytics, pricing rules and automatic repricing. [S1][S2][S3][S4][S5][S6]

The main technical challenges are not limited to price extraction. Reliable repricing also requires correct product matching, fresh observations, a consistent cost model, margin protection and reliable handling of failed collection or publishing attempts.

Some Indian D2C websites restrict automated collection in their published terms. Public visibility does not, by itself, establish permission for commercial scraping. [S16][S17]

Research finding. The product should combine reliable source-specific collection, correct product matching, explainable pricing rules and controlled publishing. A basic scraper or simple undercutting rule would not be sufficient differentiation on its own.

1.1 Research limitations

The research uses official vendor pages, published documentation, selected customer reviews, technical references and a sample of website terms.

Vendor claims have not necessarily been independently tested. Individual customer reviews are not market-wide failure rates. The research has not yet established a validated production source list or quantified customer demand through structured interviews.

2 Project Brief and Scope

2.1 Original problem statement

> **Competitor Price Scraper & Dynamic Repricer**
> A real-time competitor product pricing tracker with margin-protected pricing rules for mid-market direct-to-consumer merchants.

The research covers the relevant software category, customer workflows, existing alternatives, pricing methods, data collection and possible product opportunities.

2.2 Research questions

1. Which existing products solve this problem?
2. What do these products monitor, compare, recommend and automate?
3. What customer problems appear in public reviews?
4. How should margin-protected pricing rules work?
5. Which sources could our own scraper read?
6. What access or technical limitations apply?

7. Which customer needs and product opportunities remain unvalidated?

2.3 Evidence classification

Evidence type	Meaning
Official documentation	Published product capabilities, APIs and technical behaviour.
Vendor claim	A product capability advertised by its provider, not necessarily independently tested.
Customer-review observation	An individual experience, not evidence of the frequency of a problem across the market.
Research hypothesis	A possible customer need or opportunity that requires further validation.

3 Competitive Landscape

3.1 Existing products

Product	Published focus	Relevance
PriceSync by Jalpi	Competitor monitoring, pricing rules, price floors and store integrations.	Close match to the proposed product. [S1]
PriceZen	D2C price alerts, price history and business-impact reports.	India-focused monitoring and reporting. [S2]
Databitcent	Competitor-store monitoring, stock alerts, reports and pricing recommendations.	Useful benchmark for store-based subscriptions. [S3]
Prisync	Product matching, scheduled monitoring, analytics and dynamic repricing.	Benchmark for monitoring methods and pricing rules. [S4]
Price2Spy	Price monitoring, history, reports, integrations and optional repricing.	Benchmark for modular pricing intelligence. [S5]
PriceIntel AI	Brand-price intelligence across Indian marketplaces, including price-floor monitoring.	Adjacent solution for marketplace intelligence. [S6]

3.2 Common market capabilities

The reviewed products offer combinations of:

- Competitor price monitoring.
- Product and variant matching.
- Price history and change alerts.
- Lowest-price, average-price and custom pricing rules.
- Minimum and maximum selling-price constraints.
- Price recommendations and automatic publishing.
- Store integrations, reporting and APIs.

These capabilities are already established in the market. They should be treated as baseline capabilities rather than unique differentiators.

3.3 Customer problems reported in reviews

Selected customer reviews describe manual product matching, incorrect matches, availability uncertainty, slow competitor pages and inactive URLs. [S9][S10]

Potential areas for further validation include:

- More reliable matching between comparable products.
- Clear distinction between failed collection and genuine product unavailability.
- Visibility into the last successful price check.
- Explanations of why a price was recommended.

The available reviews do not establish the frequency of these problems across Indian D2C merchants.

4 Indian Market and Potential Customers

4.1 Potential customer segments

Segment	Possible workflow	Question to validate
Early-stage D2C founder	Manually checks a limited number of competitors during campaigns.	Is the problem frequent enough to justify paid software?
Growth-stage D2C brand	Tracks multiple products across its own website and other sales channels.	Which catalogue sizes and channels create the greatest manual effort?
Multi-channel brand team	Monitors products across marketplaces and possibly quick-commerce platforms.	Is the main need competitor repricing, brand-price compliance, or both?

A limited Indian startup community discussion described manual competitor checks and spreadsheet-based monitoring. This is qualitative evidence rather than a representative industry survey. [S20]

The working hypothesis is that growth-stage D2C merchants may have a greater need for structured price monitoring. Customer interviews are still required to validate this.

4.2 Candidate source categories

Category	Candidate platforms	Initial approach
D2C websites	Merchant-selected Shopify, WooCommerce and custom brand websites.	Closest fit to the original brief.
Indian marketplaces	Amazon.in, Flipkart, Myntra, Nykaa, Meesho, AJIO and JioMart.	Verify access and available fields.
Indian quick commerce	Blinkit, Zepto, Swiggy Instamart, BigBasket and Flipkart Minutes.	Verify access and location-specific prices.
International marketplaces	Amazon US/UK/EU, Walmart Marketplace, eBay, Etsy and AliExpress.	Potential future expansion.

This is a candidate source inventory, not a guarantee of production support. Every source requires its own access review and technical validation.

5 Pricing Strategies and Margin Protection

5.1 Common pricing strategies

Existing repricers support strategies such as:

- Undercut the lowest eligible competitor.
- Match a selected competitor's price.
- Stay a fixed amount or percentage below a competitor.
- Position against the average competitor price.
- Maintain a fixed target price.
- Enforce minimum and optional maximum prices.

PriceSync, Prisync and Price2Spy document variants of these capabilities. [S1][S4][S5]

5.2 Gross margin versus markup

Assume a product costs INR 600 and the required gross margin is 25%. If INR 600 includes every cost relevant to the defined margin calculation, the minimum selling price is:

$$P_{\min} = \frac{C}{1 - m}$$

where C is the relevant cost and m is the target gross margin as a decimal.

Therefore:

$$P_{\min} = \frac{600}{1 - 0.25} = \text{INR } 800$$

A 25% markup would instead produce a selling price of INR 750. At that price, the profit is INR 150 and the gross margin is 20%.

5.3 Competitor-pricing example

Assume the product costs INR 600, the minimum gross margin is 25%, and the competitor sells at INR 850.

A strategy that undercuts the competitor by 10% would calculate:

$$850(1 - 0.10) = \text{INR } 765$$

The minimum safe price is INR 800. The pricing engine must therefore recommend no less than INR 800 and explain why it cannot follow the competitor's price.

5.4 Additional pricing considerations

The final calculation must define the treatment of:

- Product acquisition or manufacturing cost.
- Packaging, shipping and fulfilment.
- Payment fees and marketplace commissions.
- Taxes, discounts, coupons and promotions.
- Variable selling costs charged as a percentage of the selling price.

Products must not be compared directly when currencies, pack sizes, variants or tax treatment differ. Stale observations and unavailable offers must also be handled explicitly.

6 Web-Scraping and Data Feasibility

6.1 Potential extraction methods

Method	How it works	Main limitation
HTML parsing	Extract relevant fields from the HTML returned by a product page.	Page structure may change or the price may be missing from the initial response.
Structured product data	Read product and offer fields where structured markup is available. [S11]	Data may be incomplete or stale.
Browser-rendered extraction	Render a permitted page when client-side content must be loaded.	More resource-intensive and still subject to access restrictions.
Authorised API or feed	Retrieve structured data through a documented and permitted interface.	Access, quotas and available fields vary by platform.

6.2 Potential observation fields

Where available and permitted, an observation may contain:

- Source URL and source domain.
- Product title, brand, model and identifier.
- Variant, size, colour and pack quantity.
- Displayed selling price, currency, MRP and promotional context.
- Availability and observation timestamp.
- Collection outcome and error details.

Not every source exposes all fields.

6.3 Product matching

Similar product titles do not guarantee equivalent products. Matching may require GTIN, EAN, UPC, SKU, brand, model and variant attributes.

An uncertain match must not be treated as confirmed. The system should request manual review when the available evidence is insufficient.

6.4 Failure handling and freshness

The system must distinguish:

- A successfully observed price.
- A stale observation.
- A confirmed unavailable product.
- Unknown availability.
- A failed collection attempt.

A failed request must never be interpreted as a price of zero or a confirmed stockout.

6.5 Source access restrictions

Shopify's Admin API can retrieve an authorised merchant's own products, variants, prices and inventory. [S12]

Amazon and Flipkart publish APIs for authorised seller workflows, but their existence does not prove unrestricted access to every competitor's prices. [S13][S14]

Some Indian D2C websites restrict automated collection in their published terms. Foxtale and boAt are examples whose terms restrict scraping or commercial collection of product data. [S16][S17]

Blinkit, BigBasket, Zepto and Swiggy must be reviewed under their own applicable terms and access conditions before automated collection is attempted. [S25][S26][S27][S28]

The Robots Exclusion Protocol describes crawler instructions but does not grant legal authorisation by itself. [S15]

7 Competitor Pricing and Business Models

The following figures are examples of vendor-published plans reviewed during the research. Prices and limits may change and should be reverified before commercial use.

Product	Published pricing	Billing model
PriceSync	USD 9.99, 19.99 and 39.99 per month.	Product capacity and sync-balance limits. [S1]
PriceZen	INR 999 and INR 2,499 per month.	Product limits and competitors per product. [S2]
Databitcent	INR 1,999, 5,999 and 12,000 per month.	Number of monitored competitor stores. [S3]
Prisync	The reviewed URL-based page displayed 99, 199 and 399 per month.	SKU volume, monitoring method and features. Currency requires verification. [S4]
Price2Spy	Tiered plans, with custom quotations for some offerings.	Monitored URLs, integrations and optional repricing modules. [S5]

The reviewed vendors use different billing measures: product count, competitor count, monitored URLs, successful checks and feature tiers. Subscription price alone is not a fair value comparison.

8 Research Findings and Opportunities

The research suggests several areas for further validation. None has been conclusively established as a unique market gap.

8.1 Product matching

Some reviews describe manual matching effort or incorrect product matches. [S9]

A potential improvement is to combine product identifiers, model and variant attributes, and manual confirmation when matching confidence is low.

8.2 Data freshness

Existing documentation and reviews describe collection failures and inactive URLs. [S1][S10]

A potential improvement is to show the last successful observation, source health and stale-data status.

8.3 Explainable pricing

Recommendations could explain which competitor observations and pricing rules produced the result, including any constraint that prevented a lower price.

8.4 Safe repricing

Minimum prices, approval modes and automatic publishing are already available in competitor products. [S1][S4][S5]

These are important requirements but are not unique differentiators by themselves.

8.5 Research conclusion

The market category is established. The main engineering challenges are reliable collection, correct matching, sound margin calculations and safe publishing.

The initial product should focus on a defined customer workflow, a small number of validated sources and a reliable pricing process.

9 Source Strategy and Feasibility

9.1 Merchant data and competitor data

The merchant's own catalogue and competitor prices are separate sources.

An authorised Shopify Admin API integration can retrieve the merchant's own products, variants, prices and inventory. [S12]

Amazon and Flipkart seller APIs support specified authorised workflows. They must not be assumed to provide unrestricted access to all competitor listings. [S13][S14]

CSV import is a potential starting point for the merchant's own catalogue while the write-enabled store integration is being selected.

9.2 Candidate source inventory

Source	Research observation	Initial decision
D2C websites	Candidate sources include merchant-selected Shopify, WooCommerce and custom brand websites.	Validate each domain's terms and extraction method.
Amazon.in	Seller API endpoints support specified authorised operations.	Verify competitor-data access separately.
Flipkart	Seller APIs support own-listing workflows.	Verify fields and permissions for the intended use.
Quick-commerce platforms	Blinkit, Zepto, BigBasket and Swiggy Instamart may involve location-specific prices.	Use only an authorised, permitted collection method.
International platforms	Amazon US/UK/EU, Walmart Marketplace, eBay, Etsy and AliExpress.	Consider after the initial market is confirmed.

The source registry must record domain, region, source type, permitted collection method, available fields, currency, refresh limit, adapter version, review date and operational status.

Sources must remain disabled until their permitted collection method and technical feasibility have been reviewed.

Part II
Product Requirements Document

10 Product Overview

10.1 Problem statement

D2C merchants need to understand competitor prices and decide whether their own prices remain competitive without breaching minimum profitability requirements.

Manual price comparison can consume time and lead to incorrect decisions when product variants differ, price data is stale or discounts are interpreted incorrectly.

The product will collect competitor prices from approved sources, match comparable products, calculate recommendations and support controlled publishing through an authorised merchant-store integration.

10.2 Product goals

1. Reduce manual competitor-price monitoring.
2. Collect timestamped price observations.
3. Match competitor listings to merchant products.
4. Maintain searchable price history.
5. Calculate explainable price recommendations.
6. Protect configured minimum prices and margins.
7. Support authorised, controlled price publishing.
8. Make stale data and collection failures visible.

10.3 Non-goals

The first release will not:

- Support every marketplace from day one.
- Bypass CAPTCHAs, authentication or access controls.
- Treat all similar product names as exact matches.
- Publish prices when data or rules are invalid.
- Replace the merchant's complete e-commerce or accounting platform.
- Depend on a paid competitor repricing API as the core implementation.

11 Users and Permissions

Role	Responsibilities	Permissions
Workspace Administrator	Manage users, settings and integrations.	Full workspace access, including publishing controls.
Pricing Manager	Review competitor changes and make pricing decisions.	Manage products, matches, rules, recommendations and approvals.

Role	Responsibilities	Permissions
Analyst / Viewer	Monitor prices and prepare reports.	View products, observations, history and reports. No publishing or rule changes without permission.

Publishing permissions must be enforced on the server, not only in the user interface.

12 Primary User Workflows

12.1 Workflow A: Configure monitoring

1. The user creates or opens a workspace.
2. The user imports products through CSV or connects a supported merchant store.
3. The user confirms the SKU, product identity, cost, current price, currency and margin settings.
4. The user adds competitor URLs individually or through a bulk CSV upload.
5. The system validates the URL and verifies that its source domain is approved.
6. The scraper collects data using the relevant source adapter and permitted access method.
7. The user confirms or corrects the product match.
8. The system stores the observation and schedules later checks within the source's approved limits.

12.2 Workflow B: Review a price change

1. A scheduled job collects a new observation.
2. The system validates it and compares it with the previous valid observation.
3. A configured price-change condition creates an alert.
4. The pricing engine calculates a recommendation when the match, data freshness and rule are valid.
5. The user reviews the suggested price, evidence, margin calculation and explanation.

12.3 Workflow C: Publish a price

1. The system verifies source freshness and match status.
2. It checks the price floor, ceiling, cost basis, rounding rule and change limits.
3. In approval mode, an authorised user approves the proposed update.
4. In automatic mode, publishing is permitted only after explicit activation and successful safeguards.
5. The update is sent through the authorised API of the connected merchant store.
6. The system records the request, result, old price, new price and timestamps.
7. Failed or uncertain updates remain unresolved until they can be verified.

13 Feature Scope and Priorities

P0 represents essential first-release capabilities. P1 represents subsequent improvements. P2 represents future expansion.

Priority	Capability	Scope
P0	Product catalogue	Manual entry, CSV import, SKU, cost basis, current price, currency and variants.
P0	Competitor URLs	Single URL entry, bulk import, validation and source status.
P0	Source adapters	A small number of sources that pass access review and technical testing.
P0	Price collection	Extract available fields, record timestamps, validate results and preserve failures.
P0	Product matching	Manual confirmation and identifier-assisted matching suggestions.
P0	Price history	Store and display timestamped observations.
P0	Pricing engine	Configurable strategies, price floor, optional ceiling and explanations.
P0	Safe repricing	Simulation, approval mode and one authorised publishing connector.
P0	Dashboard and alerts	Product comparisons, history, recommendations, freshness and collection failures.
P0	Security and audit	Authentication, tenant isolation, credential protection and action logging.
P1	Additional connectors	Add approved marketplace and quick-commerce sources after demand and feasibility are validated.
P1	Enhanced matching	Improve matching with product attributes and review for uncertain results.
P1	Notifications and exports	Add email or messaging alerts and CSV exports.
P2	Competitor discovery	Suggest potential listings through permitted search or catalogue interfaces.
P2	Advanced analytics	Evaluate richer pricing and regional analytics after the core workflow is validated.

14 Platform and Source Strategy

14.1 Merchant-store integrations

Merchant data and competitor data must be treated as separate workflows. Store integrations retrieve the merchant's own catalogue and may publish its prices where authorised.

Integration	Potential use	Requirement
CSV import/export	Initial catalogue setup and data exchange.	Validate data types, currencies, duplicates and missing values.
Shopify	Read products and variants; publish prices where authorised.	Use the official Admin API and required permissions. [S12][S22]
WooCommerce	Read and update the merchant's own catalogue.	Verify authentication, permissions and variable-product support.
Amazon Seller API	Retrieve eligible seller-side information.	API access is endpoint-specific and does not imply unrestricted access to competitor listings. [S13][S24]

Integration	Potential use	Requirement
Flipkart Seller API	Read or update connected seller listings.	Use seller authorisation and validate individual update results. [S14][S23]

One write-enabled merchant-store connector must be selected for the first release.

14.2 Competitor source inventory

Source category	Candidate platforms	Initial approach
D2C websites	Merchant-selected Shopify, WooCommerce and custom brand websites.	Validate a small permitted set first.
Indian marketplaces	Amazon.in, Flipkart, Myntra, Nykaa, Meesho, AJIO and JioMart.	Verify access, fields and product matching.
Indian quick commerce	Blinkit, Zepto, Swiggy Instamart, BigBasket and Flipkart Minutes.	Verify permitted access and local pricing.
International marketplaces	Amazon US/UK/EU, Walmart Marketplace, eBay, Etsy and AliExpress.	Consider after launch scope is confirmed.

The source registry must record the domain, region, source type, approved collection method, access review, available fields, currency, refresh limit, adapter version and source status.

The scraper must not bypass authentication, CAPTCHAs, access controls or anti-bot protections. A source must remain disabled unless a permitted collection method is available.

15 Functional Requirements

ID	Area	Requirement	Priority
FR-001	Workspace	Provide authenticated workspaces with role-based access to products, rules, integrations and publishing controls.	P0
FR-002	Catalogue	Allow manual product creation and validated CSV import.	P0
FR-003	Catalogue	Products shall support SKU, name, brand, identifiers, cost basis, current price, currency and variants.	P0
FR-004	Import	Report invalid rows, currencies, prices and duplicates instead of silently importing them.	P0
FR-005	Store connection	Connect to one selected authorised merchant platform to retrieve own-product data.	P0
FR-006	Store connection	Require validated credentials and write permissions before price publishing is available.	P0
FR-007	Competitor URLs	Add, edit, pause and remove competitor URLs individually or through bulk CSV import.	P0
FR-008	URL safety	Block unsafe schemes, private addresses, metadata endpoints and prohibited redirects.	P0
FR-009	Source registry	Schedule collection only for approved source domains.	P0
FR-010	Collection	Use source-specific adapters and record the adapter version with observations.	P0

ID	Area	Requirement	Priority
FR-011	Extraction	Extract supported prices, currency, product identity, variants and availability where reliable.	P0
FR-012	Validation	Flag missing prices, currency mismatches, implausible values and suspicious changes.	P0
FR-013	Scheduling	Respect approved source limits and use bounded retries for eligible transient errors.	P0
FR-014	Job status	Distinguish queued, running, successful, partial, failed, blocked and stale states.	P0
FR-015	Matching	Allow users to confirm or correct competitor matches.	P0
FR-016	Matching	A title-similarity score alone shall not confirm product equivalence.	P0
FR-017	History	Store valid observations with price, currency, source, timestamp, availability and result.	P0
FR-018	History	Allow users to view price history over a date range.	P0
FR-019	Freshness	Show the last successful observation and highlight data that exceeds the configured freshness limit.	P0
FR-020	Alerts	Support configurable alerts for price changes, availability changes, stale data and failures.	P0
FR-021	Pricing rules	Support competitor matching, undercutting, average-price positioning and manual target prices.	P0
FR-022	Pricing rules	Support a minimum price or margin floor and an optional maximum price.	P0
FR-023	Validation	Reject inconsistent floor and ceiling settings.	P0
FR-024	Pricing engine	Use valid, fresh observations and confirmed product matches for recommendations.	P0
FR-025	Explanation	Record recommendation inputs, rule version, candidate price, constraints and final result.	P0
FR-026	Simulation	Calculate recommendations without publishing prices.	P0
FR-027	Approval	Allow authorised users to approve, reject or defer recommendations.	P0
FR-028	Automatic mode	Keep automatic publishing disabled by default. Require explicit activation by an authorised user.	P0
FR-029	Safeguards	Block publishing for stale data, unconfirmed matches, invalid rules, missing permissions or floor violations.	P0
FR-030	Publishing	Record old and new prices, actor, rule, timestamps, platform response and final result.	P0
FR-031	Errors	Report collection and publishing failures accurately. Never claim an unconfirmed change succeeded.	P0
FR-032	Export	Provide product and price-history CSV exports.	P1
FR-033	Integrations	Allow new connectors to use the common observation model without changing core pricing logic.	P1
FR-034	Matching	Improve suggestions using identifiers and product attributes, with review when uncertain.	P1
FR-035	Discovery	Suggest possible competitors through permitted search interfaces, subject to user review.	P2
FR-036	Analytics	Add advanced analytics after the core workflow has been validated.	P2

16 Recommended Technology Stack

The following is a practical baseline for an independently built first release. It keeps the frontend and backend separate while allowing the scraper and pricing engine to remain under our control.

Layer	Technology	Purpose
Frontend	React, TypeScript and Vite. [S29][S30][S31]	Dashboard, catalogue, comparison screens, settings, alerts and recommendation queue.
Styling	Tailwind CSS. [S32]	Responsive layout and reusable interface styles.
Charts	Recharts. [S33]	Price history, price gaps and trends.
Backend API	Python and FastAPI. [S34]	Authentication, validation, product management, pricing services and integration endpoints.
Scraping	HTTPX, Beautiful Soup and lxml. [S35][S36][S43]	Retrieve permitted pages, parse content and normalise extracted data.
Browser rendering	Playwright when permitted pages require it. [S37]	Render selected pages where browser execution is necessary and authorised.
Background jobs	Celery and Redis. [S38][S39]	Scheduled collection, retries, imports and publishing jobs.
Database	PostgreSQL. [S40]	Products, source settings, observations, rules, recommendations, users and audit history.
ORM and migrations	SQLAlchemy and Alembic. [S41][S42]	Database models and repeatable schema migrations.
Testing	Pytest, API tests, parser fixtures and browser-based end-to-end tests. [S44]	Validate calculation accuracy, adapters, permissions, jobs and user workflows.
Deployment	Static frontend hosting and container-hosted API and worker.	Separate long-running background jobs from interactive web requests.

A modular application with a dedicated background worker is sufficient for the first release. A large microservices architecture is unnecessary without a clear scale or operational requirement.

17 System Architecture

17.1 Architecture overview

The system has five logical layers:

1. **User interface:** product management, competitor comparisons, rules and approval workflows.
2. **Backend API:** authentication, validation, workspace permissions and business operations.
3. **Collection layer:** source registry, scheduled jobs, website adapters and validation.
4. **Pricing layer:** matching, price history, pricing rules and recommendations.
5. **Publishing layer:** approval controls and authorised merchant-store updates.

17.2 System architecture diagram

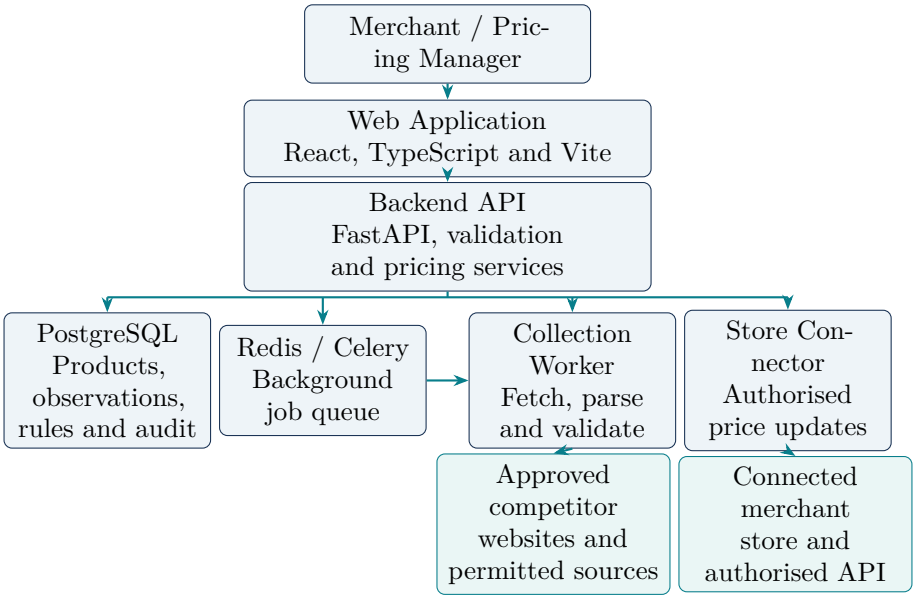


Figure 1: Proposed logical architecture for data collection, pricing and controlled publishing.

17.3 Architecture principles

- **Own the core logic:** source adapters, extraction, matching, pricing and history belong to the product.
- **Separate source adapters:** each platform uses its own extraction logic but returns data through a common interface.
- **Run collection asynchronously:** scraping, retries and store updates run as background jobs.
- **Keep observations traceable:** each price observation has a source and timestamp.
- **Centralise price calculations:** every recommendation uses the same tested pricing service.
- **Separate recommendation from publishing:** calculating a price does not automatically change a live store unless automatic publishing is enabled.
- **Make source permissions explicit:** only approved domains and authorised interfaces may be used.

17.4 Component responsibilities

Component	Responsibility
Web application	Accept user input and display products, competitor comparisons, price history, recommendations and alerts.
Backend API	Authenticate requests, validate data, enforce workspace permissions and coordinate services.
Source registry	Store domain, region, approval status, collection method, supported fields, refresh limits, adapter version and health.
Source adapters	Retrieve permitted pages or authorised API responses and extract source-specific fields.
Collection worker	Execute scheduled jobs, validate observations, retry recoverable failures and record results.
Matching service	Compare identifiers and attributes, maintain match status and request review when uncertain.
Pricing service	Apply the configured strategy, price floor, ceiling, rounding and cooldown rules.

Component	Responsibility
Store connector	Read the merchant catalogue and publish approved prices through an authorised API.
PostgreSQL	Persist products, sources, observations, matches, rules, recommendations, jobs and audit history.
Redis / Celery	Coordinate background jobs. The database remains the source of truth.

17.5 Price collection and repricing flow

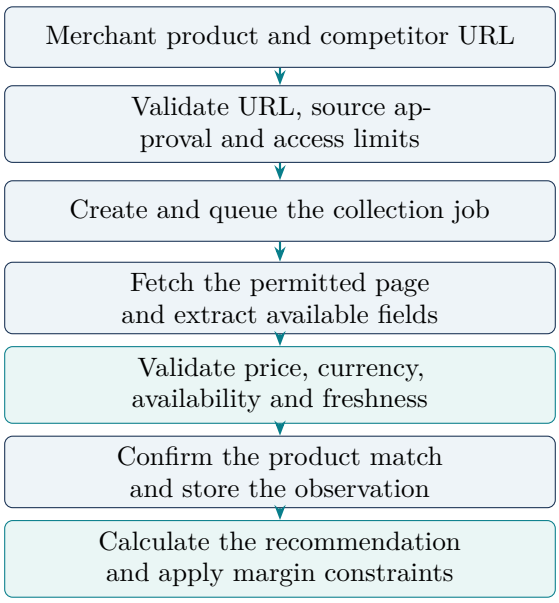


Figure 2: Price collection, validation, matching and repricing flow.

If collection, validation, matching or pricing rules fail, the system records the reason and prevents unsafe recommendations or publishing.

18 Recommended Project Structure

The frontend, backend API, scraping adapters, pricing services, background jobs and tests should have clear boundaries.

```
competitor-price-repricer/
|
|-- frontend/
|   |-- src/
|   |   |-- components/
|   |   |-- pages/
|   |   |-- features/
|   |   |   |-- products/
|   |   |   |-- competitors/
|   |   |   |-- pricing-rules/
|   |   |   |-- recommendations/
|   |   |   |-- price-history/
|   |   |-- services/
|   |   |-- types/
|   |-- public/
```

```
|   |-- package.json
|
|-- backend/
|   |-- app/
|   |   |-- api/
|   |   |-- core/
|   |   |-- models/
|   |   |-- schemas/
|   |   |-- repositories/
|   |   |-- services/
|   |   |-- connectors/
|   |   |   |-- merchant_stores/
|   |   |   |-- competitors/
|   |   |-- scraping/
|   |   |-- pricing/
|   |   |-- jobs/
|   |   |-- workers/
|   |   |-- main.py
|   |-- migrations/
|   |-- tests/
|   |-- pyproject.toml/
|
|-- deployment/
|-- docs/
|-- .env.example
|-- docker-compose.yml
|-- README.md
```

Secrets and real credentials must never be committed to source control. The example environment file should contain variable names and safe placeholders only.

19 Deployment Architecture

The recommended deployment separates the frontend, API, collection worker, scheduler, database and job queue while avoiding unnecessary microservices.

Component	Deployment option	Considerations
Frontend	Vercel or equivalent static hosting.	Serve the React application over HTTPS and configure the backend API origin.
Backend API	Container hosting such as Render or an equivalent managed service.	Health checks, environment configuration, structured logs and request timeouts.
Collection worker	Separate worker process or container using the same backend codebase.	Independent operation, bounded concurrency and source-specific limits.
Scheduler	Worker scheduler or a dedicated scheduled process.	Create jobs at the permitted interval for each source.
Database	Managed PostgreSQL.	Backups, migrations, access control and connection limits.
Queue	Managed Redis or equivalent supported service.	Restrict access and monitor queue depth and failures.

Component	Deployment option	Considerations
Secrets	Environment configuration or managed secret storage.	Keep API keys and OAuth credentials out of code and logs.

The final hosting provider should be selected after catalogue size, background-job volume, browser requirements and budget are known. The frontend host should not run long-lived scraping tasks.

19.1 Environment variables

Variable	Purpose
`DATABASE_URL`	PostgreSQL connection.
`REDIS_URL`	Background job broker.
`APP_SECRET_KEY`	Application signing or session secret.
`FRONTEND_ORIGIN`	Approved frontend origin.
`SHOPIFY_CLIENT_ID`	OAuth client ID if Shopify is selected.
`SHOPIFY_CLIENT_SECRET`	Private OAuth client secret.
`STORE_OAUTH_REDIRECT_URI`	Approved OAuth callback URL.
`LOG_LEVEL`	Logging configuration.

Variables for other platforms should be introduced when those integrations are selected. Secret values must not appear in dashboard responses or error logs.

20 Proposed Data Model

Entity	Purpose and important fields
Workspace	Tenant boundary, name, settings and timestamps.
User and Membership	User identity, workspace role, permissions and status.
Store Connection	Platform, external store ID, encrypted credential reference, granted scopes, status and last sync.
Product	Workspace, SKU, name, brand, product identifiers, cost basis, current price, currency and status.
Product Variant	Parent product, external identifier, size, colour, options, pack quantity and attributes.
Source Domain	Domain, region, approved collection method, access review, supported fields, refresh limits, adapter version and status.
Competitor Listing	Source domain, URL, extracted identity, variant, currency, match status and monitoring status.
Product Match	Merchant product, competitor listing, matching method, review state, reviewer and timestamps.
Price Observation	Listing, price, currency, MRP or discount where available, availability, timestamp and collection result.
Scrape Job	Source URL, schedule, start/end times, status, retry count, adapter version and error category.
Pricing Rule	Strategy, competitor set, cost basis, margin, floor, ceiling, rounding, minimum change and version.

Entity	Purpose and important fields
Price Recommendation	Product, rule version, observation IDs, candidate price, constraints, final price, explanation and status.
Publishing Job	Store connection, SKU, old/new price, approval, idempotency key, platform response and timestamps.
Alert	Alert type, product/listing, trigger observation, recipient, status and delivery result.
Audit Event	Actor, workspace, action, affected entity, relevant old/new values, result and timestamp.

All tenant-owned records must be associated with a workspace. Database queries, APIs, background jobs and exports must prevent cross-workspace data access.

21 API Requirements

The backend should expose a documented REST API under `/api/v1/`. Exact endpoint names can be refined during implementation.

Method	Resource	Purpose	Priority
POST	`/products/import`	Import and validate a product CSV.	P0
GET	`/products`	Search and list workspace products.	P0
POST	`/store-connections`	Register or initiate an authorised store connection.	P0
GET	`/store-connections/status`	View connection health and permissions.	P0
POST	`/competitor-listings`	Add competitor product URLs.	P0
POST	`/competitor-listings/import`	Bulk-import and validate competitor URLs.	P0
POST	`/product-matches/confirm`	Confirm or correct product matches.	P0
GET	`/price-observations`	Retrieve observations and price history.	P0
POST	`/monitoring-jobs`	Schedule a permitted collection job.	P0
GET	`/monitoring-jobs/status`	Inspect job status and failures.	P0
POST	`/pricing-rules`	Create or update a pricing rule.	P0
GET	`/recommendations`	Retrieve recommendations and explanations.	P0
POST	`/recommendations/approve`	Approve a recommendation for publishing.	P0
POST	`/price-publishing`	Submit an approved price through the authorised store connector.	P0
GET	`/alerts`	List price-change, freshness and failure alerts.	P0
GET	`/audit-events`	Retrieve audit history.	P0

Every endpoint must validate authentication, workspace access, input data and permissions. Publishing and rule-changing endpoints must apply role checks and create audit events.

22 Pricing Engine Requirements

22.1 Calculation inputs

The pricing engine may use:

- Product cost and its defined cost basis.
- Current selling price and currency.
- Target gross margin or explicit minimum price.
- Fixed and variable selling costs where applicable.
- Eligible competitor observations and freshness.
- Product-match and availability status.
- Strategy, ceiling, rounding and minimum-change rules.

The cost basis must define tax treatment, shipping, fulfilment, marketplace fees, payment fees, packaging and discounts. Cost and revenue must use a consistent basis.

22.2 Minimum safe selling price

For fixed relevant cost C, without additional percentage-based selling costs:

$$P_{\min} = \frac{C}{1 - m}$$

where m is the target gross margin as a decimal.

If variable selling costs consume a proportion r of the selling price, and the desired margin is measured after those costs:

$$P_{\min} = \frac{C}{1 - r - m}$$

The second formula is valid only when $1 - r - m > 0$ and the cost basis is consistent. Invalid inputs must be rejected instead of producing a recommendation.

22.3 Recommendation algorithm

The pricing engine shall:

1. Confirm that merchant and competitor matches are valid.
2. Check the freshness and validity of observations.
3. Exclude failed, stale, currency-incompatible and unconfirmed observations.
4. Exclude unavailable offers where the selected rule requires an available competitor.
5. Calculate the base target from the selected strategy and eligible competitor set.
6. Calculate the minimum price from the configured margin and any stricter explicit floor.
7. Apply the optional ceiling and rounding rule. Rounding must never lower the price below the floor.
8. Block the recommendation if the floor exceeds the ceiling or no eligible observations remain.
9. Apply configured minimum-change or cooldown rules.
10. Save the inputs, rule version, calculated values and explanation.

22.4 Publishing safeguards

Before publishing, the system must confirm:

- The store connection and required write permissions are active.
- The recommendation is within the configured floor and ceiling.

- Observations are valid and sufficiently fresh.
- The product match is confirmed.
- Cost inputs and pricing rules are complete.
- Automatic publishing is enabled or an authorised user has approved the recommendation.
- The same update has not already been submitted.

Publishing results must be recorded per SKU or variant. Partial success must not be reported as total success. [S12][S14]

23 Dashboard and Interface Requirements

Screen	Required content and actions
Overview Dashboard	Active products, monitored listings, price movements, pending recommendations, data freshness and failures.
Product Catalogue	Searchable products, SKU, current price, cost basis, price floor, monitoring status and import errors.
Competitor Mapping	Merchant product, competitor listing, variant attributes, source, match status and review controls.
Product Detail and History	Merchant price, competitor observations, history, availability, last successful check and source links.
Pricing Rules	Strategy, competitors, margin, floor, ceiling, rounding and activation state.
Recommendation Queue	Current and suggested prices, evidence, margin, explanation, blocked reason, approval controls and publishing state.
Source Monitoring	Source approval, latest job, next run, failures, source health and pause/resume controls.
Integrations	Connected store, permissions, connection health and publishing capability.
Audit History	Rule changes, approvals, updates, failures, actors and timestamps.

The interface must be responsive and accessible. Loading, empty, error, blocked and stale-data states must be designed explicitly. Users must be able to tell whether a price is current and whether an update reached the store.

24 Non-Functional Requirements

Area	Requirement
Security	Encrypt credentials at rest, use TLS, keep secrets out of source control and apply least privilege.
Tenant isolation	Prevent cross-workspace access in APIs, database queries, background jobs and exports.
URL safety	Validate schemes, block private and metadata addresses, revalidate redirects, constrain ports and response sizes, and mitigate SSRF.
Source access	Respect source terms, API permissions, rate limits and approved collection intervals. Do not bypass access or anti-bot controls.

Area	Requirement
Reliability	Use bounded retries, timeouts, idempotent jobs, visible failures and safe partial-result handling.
Data integrity	Store source, timestamp, currency, match status and collection result with each observation.
Auditability	Record rule changes, match confirmations, approvals, publishing attempts and failures.
Observability	Monitor job completion, extraction errors, stale observations, publishing failures and source health.
Performance	Use pagination and background jobs for large imports and monitoring tasks.
Accessibility	Provide readable text, semantic controls, keyboard navigation and clear error messages.
Data retention	Define retention periods for price history, logs and source evidence. Retain only necessary and permitted data.

Performance and availability targets should be set after source tests establish realistic timings and reliability.

25 Security and Operational Controls

25.1 Authentication and authorisation

The application must authenticate users and enforce workspace-level permissions on the server. Store integrations must use supported authorisation flows and request only required scopes.

25.2 URL and network security

User-supplied URLs create server-side request forgery risks. The system must validate destinations, block local and private network addresses, validate redirects and prevent access to internal services or cloud metadata endpoints.

25.3 Credential handling

Store API tokens and application secrets must be encrypted and restricted to the services that need them. Credentials must not appear in logs, error messages, browser storage or source control.

25.4 Operational monitoring

The system should track:

- Collection success and failure rates.
- Stale observations and source health.
- Job queue delay.
- Adapter and parsing errors.
- Recommendations blocked due to invalid data.
- Store update outcomes and unresolved failures.

26 Acceptance Criteria

ID	Acceptance criterion
AC-01	A valid product CSV imports correctly, while invalid rows receive clear row-level error messages.
AC-02	A successful collection stores the price, currency, source URL, timestamp and collection status.
AC-03	Unsafe URLs and unapproved sources are rejected before collection is scheduled.
AC-04	Failed collection is recorded as a failure and never becomes a zero price or confirmed stockout.
AC-05	Ambiguous variants require review before the match is confirmed.
AC-06	A relevant cost of INR 600 and target gross margin of 25% produce a minimum price of INR 800.
AC-07	A competitor-based target below the price floor is constrained to the floor, with an explanation.
AC-08	An invalid floor/ceiling combination is rejected and produces no publishable recommendation.
AC-09	Stale or invalid observations cannot produce an automatically publishable recommendation.
AC-10	Simulation mode calculates recommendations without changing a live store price.
AC-11	Approval mode blocks publication until an authorised user approves the recommendation.
AC-12	Automatic publishing is blocked when required permissions, current data, confirmed matching or valid rules are missing.
AC-13	Publishing results are recorded per item. Unconfirmed or failed updates are not reported as successful changes.
AC-14	Rule changes and price updates record the actor, relevant values, timestamps and outcomes.
AC-15	Users cannot access another workspace's products, observations, rules or audit data.

Floor protection, approval, tenant isolation and audit criteria must pass automated tests before live automatic repricing is enabled.

27 Product Metrics

Targets should be established after permitted sources have been tested and the initial workflow has been validated.

Metric	Definition
Collection success rate	Successful valid observations divided by attempted checks on approved sources.
Price extraction accuracy	Correctly extracted prices divided by audited extracted prices in a representative test set.
Product-match precision	Correct automatic matches divided by audited automatic matches.
Freshness compliance	Active listings with a valid observation inside the configured freshness window.
Recommendation acceptance rate	Recommendations accepted divided by recommendations reviewed.

Metric	Definition
Price-floor protection	All defined tests that attempt floor-violating publishing must be blocked.
Publishing success rate	Successful individual price updates divided by valid authorised publishing attempts.
Time to alert	Time between a valid detected price change and alert creation.
Source maintenance burden	Frequency of adapter failures requiring manual intervention.

28 Risks and Mitigations

Risk	Potential impact	Mitigation
Source restrictions	Collection may not be permitted or may stop.	Maintain source approvals and disable unapproved sources.
Page layout changes	Extraction fails or reads incorrect values.	Use source-specific adapters, validation and monitoring.
Wrong product matches	Misleading comparisons and unsafe recommendations.	Use identifiers and attributes, with review when uncertain.
Stale observations	Incorrect pricing decisions.	Enforce freshness limits and block unsafe publishing.
Price wars	Repeated reductions and margin erosion.	Apply floors, rounding, minimum-change thresholds, cooldowns and approval controls.
Incorrect cost inputs	The floor does not protect the intended margin.	Define and validate the cost basis.
Failed store updates	Store price differs from the intended price.	Record individual results and verify updates where supported.
Excessive initial scope	Delayed delivery and unreliable connectors.	Start with one store connector and a small set of approved competitor sources.

29 Implementation Sequence

1. **Foundation:** workspace authentication, product catalogue, CSV import, source registry, URL validation and audit logging.
2. **Data collection:** first approved adapters, scheduled jobs, extraction validation, failure handling and price history.
3. **Matching and pricing:** product mapping, pricing rules, margin calculations, explanations and simulation mode.
4. **Dashboard and alerts:** catalogue, comparison, history, recommendations, freshness and failure notifications.
5. **Publishing pilot:** one authorised merchant-store integration, starting with approval-based updates.
6. **Automatic repricing:** enable only after source reliability, margin protection, audit and publishing tests pass.

7. **Expansion:** add marketplaces and quick-commerce sources after demand, access and maintenance effort have been validated.

30 Decisions Required Before Implementation

Decision	Why it matters	Status
First store connector	Determines catalogue import and live price publishing.	Open
First competitor sources	Determines the first adapters and testing fixtures.	Open
Target product categories	Determines product identity and variant matching.	Open
Launch geography	Determines currency, sources and applicable terms.	Open
Cost and margin definition	Determines the price-floor calculation.	Open
Monitoring interval	Must fit customer needs and source limits.	Open
Automatic publishing policy	Determines approval and safety controls.	Open
Initial catalogue size	Affects storage, scheduling and pilot testing.	Open
Notification channel	Determines in-app, email or messaging requirements.	Open

A Source Register

The sources below support the research and proposed stack. Vendor pages describe their own products and are treated as vendor claims unless independently tested. Prices, features, API details and terms may change.

1. **PriceSync by Jalpi: product and pricing**
   https://pricesync.jalpi.com/
2. **PriceZen: D2C price monitoring**
   https://pricezen.in/
3. **Databitcent: competitor monitoring**
   https://www.databitcent.com/
4. **Prisync: pricing plans and monitoring**
   https://prisync.com/compare-plans/
5. **Price2Spy: pricing and repricing**
   https://www.price2spy.com/pricing.html
6. **PriceIntel AI: Indian price intelligence**
   https://priceintel.ai/solutions/india-brands
7. **Prisync: URL, channel and hybrid monitoring**
   https://helpcenter.prisync.com/hc/en-us/articles/23670383315740-URL-Channel-or-Hybrid-What-s-the-Difference
8. **Price2Spy: API reference**
   https://www.price2spy.com/api/documentation/api-reference-guide.html
9. **G2: Prisync customer reviews**
   https://www.g2.com/products/prisync/reviews
10. **G2: Price2Spy customer reviews**
    https://www.g2.com/products/price2spy/reviews
11. **Google Search Central: Product structured data**
    https://developers.google.com/search/docs/appearance/structured-data/product

12. **Shopify Admin API: products query**
https://shopify.dev/docs/api/admin-graphql/latest/queries/products

13. **Amazon Selling Partner API**
https://developer-docs.amazon.com/sp-api/

14. **Flipkart Seller API**
https://seller.flipkart.com/api-docs/index.html

15. **IETF RFC 9309: Robots Exclusion Protocol**
https://www.rfc-editor.org/rfc/rfc9309.html

16. **Foxtale: Terms of Service**
https://foxtale.in/pages/terms-of-service

17. **boAt: Terms of Use**
https://www.boat-lifestyle.com/pages/terms-of-use

18. **Ricoz Innovation: public product directory**
https://innovation.ricoz.in/products

19. **Wiser Solutions: price monitoring**
https://www.wiser.com/products/price-execution/price-monitoring/

20. **Indian startup community discussion**
https://www.reddit.com/r/IndiaStartups/

21. **Intelligence Node: retail data and APIs**
https://www.intelligencenode.com/solutions/apis-data/

22. **Shopify: product variant bulk update**
https://shopify.dev/docs/api/admin-graphql/latest/mutations/productVariantsBulkUpdate

23. **Flipkart: update listing price**
https://seller.flipkart.com/api-docs/listing-api-docs/v3/UpdateListingsPriceAPI.html

24. **Amazon: competitive pricing endpoint**
https://developer-docs.amazon.com/sp-api/lang-en_US/reference/getcompetitivepricing

25. **Blinkit: Terms**
https://blinkit.com/terms

26. **BigBasket: Terms and conditions**
https://www.bigbasket.com/terms-and-conditions/

27. **Zepto: Terms of service**
https://www.zepto.com/s/terms-of-service

28. **Swiggy: Terms and conditions**
https://www.swiggy.com/terms-and-conditions

29. **React documentation**
https://react.dev/

30. **TypeScript documentation**
https://www.typescriptlang.org/docs/

31. **Vite documentation**
https://vite.dev/guide/

32. **Tailwind CSS documentation**
https://tailwindcss.com/docs

33. **Recharts documentation**
https://recharts.org/en-US/

34. **FastAPI documentation**
https://fastapi.tiangolo.com/

35. **HTTPX documentation**
https://www.python-httpx.org/

36. **Beautiful Soup documentation**
https://www.crummy.com/software/BeautifulSoup/bs4/doc/

37. **Playwright Python documentation**
https://playwright.dev/python/docs/intro

38. **Celery documentation**
https://docs.celeryq.dev/

39. **Redis documentation**
https://redis.io/docs/latest/

40. **PostgreSQL documentation**
https://www.postgresql.org/docs/

41. **SQLAlchemy documentation**
https://docs.sqlalchemy.org/

42. **Alembic documentation**
https://alembic.sqlalchemy.org/en/latest/

43. **lxml documentation**
https://lxml.de/

44. **Pytest documentation**
https://docs.pytest.org/